PROJECT: MODELING CAR INSURANCE CLAIM OUTCOMES

datalab

Insurance companies invest a lot of time and money into optimizing their pricing and accurately estimating the likelihood that customers will make a claim. In many countries insurance it is a legal requirement to have car insurance in order to drive a vehicle on public roads, so the market is very large!

(
`Source: https://www.accenture.com/_acnmedia/pdf-84/accenture-machine-leaning-insuran`
)

Knowing all of this, On the Road car insurance have requested your services in building a model to predict whether a customer will make a claim on their insurance during the policy period. As they have very little expertise and infrastructure for deploying and monitoring machine learning models, they've asked you to identify the single feature that results in the best performing model, as measured by accuracy, so they can start with a simple model in production.

They have supplied you with their customer data as a csv file called `car_insurance.csv`, along with a table detailing the column names and descriptions below.

The dataset

Column	Description
`id`	Unique client identifier
`age`	Client's age: • `0` : 16-25 • `1` : 26-39 • `2` : 40-64 • `3` : 65+
`gender`	Client's gender: • `0` : Female • `1` : Male
`driving_experience`	Years the client has been driving: • `0` : 0-9 • `1` : 10-19 • `2` : 20-29 • `3` : 30+
`education`	Client's level of education: • `0` : No education • `1` : High school • `2` : University
`income`	Client's income level: • `0` : Poverty • `1` : Working class • `2` : Middle class • `3` : Upper class
`credit_score`	Client's credit score (between zero and one)
`vehicle_ownership`	Client's vehicle ownership status: • `0` : Does not own their vehilce (paying off finance) • `1` : Owns their vehicle
`vehcile_year`	Year of vehicle registration: • `0` : Before 2015 • `1` : 2015 or later
`married`	Client's marital status:

Column	Description
	• `0` : Not married • `1` : Married
`children`	Client's number of children
`postal_code`	Client's postal code
`annual_mileage`	Number of miles driven by the client each year
`vehicle_type`	Type of car: • `0` : Sedan • `1` : Sports car
`speeding_violations`	Total number of speeding violations received by the client
`duis`	Number of times the client has been caught driving under the influence of alcohol
`past_accidents`	Total number of previous accidents the client has been involved in
`outcome`	Whether the client made a claim on their car insurance (response variable): • `0` : No claim • `1` : Made a claim

```python
# Import required modules
import pandas as pd
import numpy as np
from statsmodels.formula.api import logit

# Read in dataset
cars = pd.read_csv("car_insurance.csv")

# Check for missing values
cars.info()

# Fill missing values with the mean
cars["credit_score"].fillna(cars["credit_score"].mean(), inplace=True)
cars["annual_mileage"].fillna(cars["annual_mileage"].mean(), inplace=True)

# Empty list to store model results
models = []

# Feature columns
features = cars.drop(columns=["id", "outcome"]).columns

# Loop through features
for col in features:
    # Create a model
    model = logit(f"outcome ~ {col}", data=cars).fit()
    # Add each model to the models list
    models.append(model)

# Empty list to store accuracies
accuracies = []

# Loop through models
for feature in range(0, len(models)):
    # Compute the confusion matrix
    conf_matrix = models[feature].pred_table()
    # True negatives
    tn = conf_matrix[0,0]
    # True positives
    tp = conf_matrix[1,1]
    # False negatives
    fn = conf_matrix[1,0]
    # False positives
    fp = conf_matrix[0,1]
    # Compute accuracy
    acc = (tn + tp) / (tn + fn + fp + tp)
    accuracies.append(acc)
```

```python
# Find the feature with the largest accuracy
best_feature = features[accuracies.index(max(accuracies))]

# Create best_feature_df
best_feature_df = pd.DataFrame({"best_feature": best_feature,
                                "best_accuracy": max(accuracies)},
                                index=[0])
best_feature_df
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   id                   10000 non-null  int64
 1   age                  10000 non-null  int64
 2   gender               10000 non-null  int64
 3   driving_experience   10000 non-null  object
 4   education            10000 non-null  object
 5   income               10000 non-null  object
 6   credit_score         9018 non-null   float64
 7   vehicle_ownership    10000 non-null  float64
 8   vehicle_year         10000 non-null  object
 9   married              10000 non-null  float64
 10  children             10000 non-null  float64
 11  postal_code          10000 non-null  int64
 12  annual_mileage       9043 non-null   float64
 13  vehicle_type         10000 non-null  object
 14  speeding_violations  10000 non-null  int64
 15  duis                 10000 non-null  int64
 16  past_accidents       10000 non-null  int64
 17  outcome              10000 non-null  float64
```

	best_feature	best_a...
0	driving_experience	0.7771

Rows: 1 Expand